

P3 - Sistema domótico

1. Introducción

En esta práctica se va a implementar un sistema domótico para el hogar con dispositivos IoT.

Un sistema de domótica de hogar permite conectar distintos tipos de dispositivos como interruptores, sensores o motores para realizar acciones cotidianas como encender una luz, medir una temperatura o subir una persiana. Estos sensores y actuadores se usan en conjunto con reglas que permiten hacer cosas como: "encender la luz de la puerta al anochecer", "cerrar las persianas al pulsar un botón", "encender la caldera si la temperatura baja de 21 grados", etc.

Existen múltiples protocolos para comunicación IoT abiertos e incluso algunos fabricantes implementan el suyo propietario, pero en este caso nos centraremos en MQTT, de los más usados.

Se proponen dos opciones para que el usuario interactúe con el sistema:

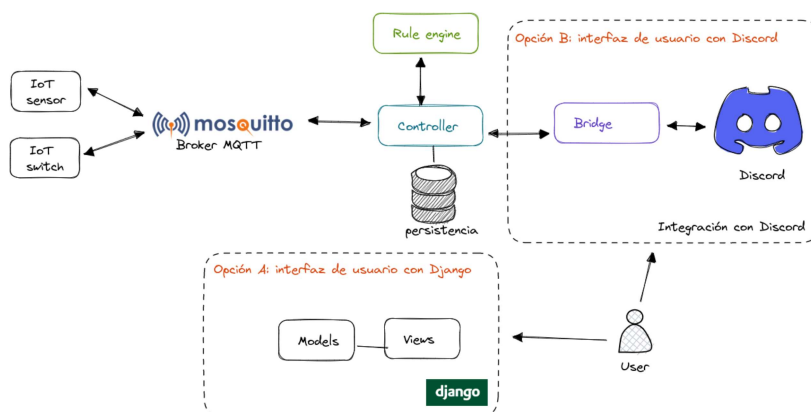
- Opción A, Django

Proyecto de Django que permita gestionar dispositivos, reglas y eventos generados.

- Opción B, Discord

Desarrollar un bot de Discord que permita, a través de mensajes publicados en una sala y leyendo acciones sobre el bot, actuar sobre los dispositivos, reglas y eventos del sistema.

En el siguiente esquema se pueden observar los distintos componentes que lo forman:



Consta de varios componentes que se explicarán en detalle más adelante:

- Dispositivos IoT
 - Sensores
 - Interruptores

- Relojes

- Broker MQTT Mosquitto
- Controller Gestión del sistema general: lee mensajes del broker, llama al Rule engine y realiza acciones sobre los dispositivos a través del broker.

- Persistencia

Almacena la información de los dispositivos registrados y las reglas.

- Rule engine

Entidad que, ante un evento, comprueba las reglas y lanza acciones

- Bridge

(Sólo en el caso de escoger la opción B)

Aplicación puente que comunica la aplicación web con Discord

- Aplicación de gestión

(Sólo en el caso de escoger la opción A)

Aplicación web basada en Django para gestionar los dispositivos, su estado

- Discord

Es una plataforma que soporta, entre otras cosas, la creación de salas de chat de texto, vídeo y audio

Y los siguientes actores:

- Dispositivo IoT (device)

Publican sus cambios de estado y pueden recibir acciones a realizar usando MQTT

- Controlador (controller)

Recibe y envía mensajes a los dispositivos IoT usando MQTT. Permite gestionar los dispositivos

- Traductor (bridge)

Recibe eventos y los traduce a acciones para el bot. Recibe acciones del bot y las traduce a acciones sobre los dispositivos.

- Bot

Publica mensajes en Discord y recibe acciones

Dispositivo IoT

Se van a soportar de tres tipos, interruptores, sensores y relojes, para los que habrá que implementar al menos uno de cada tipo.

Para cada funcionalidad que soporta un dispositivo se define un topic y las cadenas de los distintos estados. Un ejemplo para un interruptor:

- topic: `home/climate/boiler_switch`
 - state_on: `ON`
 - state_off: `OFF`
- command_topic: `home/climate/boiler_switch/set`
 - payload_on: `ON`
 - payload_off: `OFF`
- ...

Si se quiere consultar el estado del interruptor se publicará un mensaje en topic y el dispositivo responderá con uno de los estados. Si se quiere cambiar el estado se publicará un mensaje en el topic `command_topic` con el payload correspondiente y el dispositivo responderá con el estado al que ha cambiado.

Para los sensores, el dispositivo puede publicar cada cierto tiempo (o con cada cambio) el cambio de estado y lo publica en su topic. Es posible también preguntar por su estado como en el ejemplo anterior.

Controlador

Es un suscriptor y publicador en MQTT para recibir los cambios de estado y realizar cambios en los dispositivos. Persiste los cambios en los dispositivos y genera eventos.

Permite recibir acciones a realizar sobre los dispositivos IoT

Rule engine

Permite gestionar las reglas del sistema, recibir eventos y comprobar las reglas para realizar acciones

Opción Django

Proyecto Django que presenta vistas para gestionar los dispositivos, reglas y consultar los eventos generados.

Se puede usar la parte de persistencia que viene con Django, los modelos, como persistencia para todo el proyecto.

No es necesario que implemente autenticación ni que las vistas tengan una apariencia determinada mediante CSS.

Opción Discord

Bridge

Hace de traductor entre la integración con Discord y el sistema. Estará conectado con Discord para poder recibir acciones y para publicar cambios de estado.

Bot

Recibe los mensajes de acciones y publica mensajes en la sala para la que está configurada.

Broker MQTT

Usaremos Mosquitto, el broker MQTT más ligero y popular

Dudas frecuentes

- ¿Qué persistencia se recomienda usar?

Si se usa Django, lo más sencillo es usar SQLite, que lo soporta de serie.

Aunque se opte por la opción Discord, se podría usar sólo la capa de persistencia de Django con SQLite. Otra opción es usar directamente ficheros para almacenar la información en un formato a definir por el alumno: json, yaml, etc.

- ¿Tiene que haber algún prefijo en los topics a usar?

Sí, al usar un broker compartido cada dispositivo tendrá un topic con la forma:

`redes2/GRUPO/PAREJA/ID`

Donde:

- `redes2/` Es fijo
- `GRUPO` Son los 4 dígitos del grupo al que pertenece el alumno p.ej. 2303
- `PAREJA` Son los dos dígitos de la pareja p.ej. 01
- `ID` Identificador del dispositivo

Recomendaciones de desarrollo

- Entender bien el alcance del proyecto y escribirlo: requisitos y casos de uso
 - Resolver dudas de diseño
 - Documentar el diseño
 - Pensar en qué se ha de probar para que cumplan con los casos de uso: si el sistema tiene que hacer A, ¿cómo me aseguro de que es así de manera reproducible?
 - Reutilizar lo más posible código común encapsulado en clases:
 - Conexión con el broker
 - ¿Clase abstracta, funciones comunes?
 - Recepción y envío de mensajes
 - Persistencia
 - Empezar probando desde lo más pequeño e ir ampliando
- En los ejemplos anteriores se prueban primero las partes de conexión con el broker, luego recepción y envío de mensajes, clases que los usan, etc
- Ante un comportamiento no esperado, volver atrás en las pruebas hasta descubrir qué cambio ha provocado el error. Si se tienen pruebas es fácil volver atrás y determinar el origen

Implementación

TODO: Completar por el alumno

Conclusiones

TODO: Completar por el alumno

Actividad previa

Examen - Práctica 2 (2313-2393)

Ir a...

Siguiente actividad

Examen - Práctica 3

[Política de privacidad](#)

[Contacto](#)

[Aviso legal](#)

[Accesibilidad](#)

[UAM Covid 19](#)